



南京大學

本科畢業論文

学 院	智能软件与工程学院
专 业	软件工程（智能化软件）
题 目	实践中的理论最优哈希表
年 级	2022 学 号 221900323
学生姓名	纪泽操
指导教师	刘明谋 职 称 副教授
提交日期	2026 年 5 月 20 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：实践中的理论最优哈希表

学院：智能软件与工程学院

专业：软件工程（智能化软件）

本科生姓名：纪泽操

指导教师（姓名、职称）：刘明谋 副教授

摘要：

开放寻址哈希表在工程里很常见，但在高负载下既要省内存、又要控制插入删除的扰动，并不容易。Bender 等人的论文《On the Optimal Time/Space Tradeoff for Hash Tables》给出了一条理论路线：用有限次“踢键”换更紧凑的表示。本毕业设计以该论文为主要参照，用 C++ 写了一个可运行的原型：键先经可逆置换再分区；每个 Facility 内用 Router 做局部定位；槽里做商压缩，变长恢复信息放在 MiniArray；扩缩容采用新旧两张表逐步搬迁。

实现上对原文有多处简化：k-kick 只在单个 tail cubby 里用有界随机探针和踢出链近似；MiniArray 与 Router 在更新时允许整段重打包或重建 trie，以保证代码清晰和指标好记。功能测试覆盖插入、查询、删除和重复插入；另有独立微基准查看热路径延迟。实验表明，在默认参数下查询延迟稳定在纳秒量级，插入阶段因重打包和扩容会慢一些。本工作重在把论文里的主要模块串起来并说明与理论的差距，而不是证明渐近最优性。

关键词：哈希表；时间空间权衡；探针复杂度；增强开放寻址；k-kick tree；元数据编码；动态可扩容；succinct 数据结构

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: On the Optimal Time/Space Tradeoff for Hash Tables

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software Engineering (Intelligent Software)

UNDERGRADUATE: Ji Zecao

MENTOR: Liu Mingmou, Associate Professor

ABSTRACT:

This undergraduate thesis implements a simplified prototype of the hash table construction in “On the Optimal Time/Space Tradeoff for Hash Tables.” The code uses a reversible permutation for routing, per-facility routers to locate cubbies and slots, quotient-compressed payloads with variable-length metadata, and two-table incremental resizing.

Several parts of the theory are only approximated: k-kick style behavior is reduced to a bounded kick chain inside the tail cubby, and metadata structures favor straightforward updates over the tightest asymptotics. Functional tests check correctness; a small micro-benchmark reports latency and counters. The goal is a working, observable system and a clear statement of engineering tradeoffs, not a full reproduction of every formal guarantee in the paper.

KEYWORDS: hash tables; time–space tradeoff; probe complexity; enhanced open addressing; k-kick tree; metadata encoding; dynamic expansion; succinct data structures

目 录

目 录	III
第一章 引言	1
1.1 研究背景及意义	1
1.2 国内外研究现状	1
1.3 研究路线	2
第二章 问题模型与理论基础	3
2.1 探针复杂度与球槽分配模型	3
2.2 k-kick 树分层驱逐思想	3
2.3 紧凑元数据存储核心思想	4
2.4 分层架构整体设计思想	4
2.5 商压缩的信息论最优思想	5
2.6 算法层次全景总结	5
第三章 系统设计与实现	6
3.1 整体结构	6
3.2 地址拆分与可逆置换	7
3.3 MiniArray: 按槽变长元数据	7
3.4 Router: 桶内键到槽位映射	8
3.5 商压缩、写入与校验	8
3.6 插入主路径与工程版 k-kick	9
3.7 查询与删除	10
3.8 扩缩容与双表渐进迁移	10
3.9 参数与指标口径	11

第四章 系统验证与实验设计	12
4.1 功能正确性测试	12
4.2 性能基准	12
4.3 指标与数据库观测	13
4.4 可视化与结果呈现	14
第五章 总结与展望	15
5.1 工作总结	15
5.2 不足与局限	15
5.3 可改进方向	15
参考文献	16
致 谢	17

第一章 引言

1.1 研究背景及意义

哈希表是一种重要的数据结构，在数据库系统、操作系统、编译器以及各种键值存储系统中具有广泛应用。通过哈希函数将关键字映射到数组位置，从而实现高效的数据查询、插入和删除操作。长期以来，哈希表研究的核心问题之一是时间复杂度与空间复杂度之间的权衡问题。传统哈希表在保证常数时间查询的情况下，往往需要通过降低负载因子（即增加未使用存储空间）来减少冲突，从而保证查询效率。论文 *On the Optimal Time/Space Tradeoff for Hash Tables* 指出，在常数时间操作的前提下，每个元素通常还需要额外的空间开销^[1]。该文同时给出一种新的哈希表结构，证明在保持查询 $O(1)$ 时间复杂度的同时，可将插入与删除的时间复杂度由 $O(1)$ 放宽到 $O(k)$ ，从而使每个元素的额外空间开销由 $O(\log \log n)$ 比特降至 $O(\log^{(k)} n)$ 比特¹²，在时间与空间之间得到更优的权衡。对该理论结构加以实现并在实际系统中开展实验验证，既有助于加深对哈希表理论的理解，也有助于考察其在数据库系统等场景中的应用价值。

1.2 国内外研究现状

哈希表是计算机科学中研究历史较长的数据结构之一，国内外学者提出了多种不同的实现方法。早期的哈希表主要包括链地址法和开放寻址法等方法：结构简单、实现容易，在实际系统中得到了广泛应用，但在高负载情况下容易出现性能下降等问题^[2]。随后，研究者提出了多种改进算法，例如布谷鸟哈希^[3]和完美哈希^[4]，在提高查询效率或空间利用率方面取得了一定成果。近年来，研究重点逐渐转向哈希表的空间效率问题，许多理论研究尝试逼近哈希表的信息论空间下界。在此背景下，论文 *On the Optimal Time/Space Tradeoff for Hash Tables* 提

1 本文中讨论的空间开销均以比特为单位。

2 记号 $\log^{(k)} n$ 表示对 n 连续取 k 次对数： $\log \log \cdots \log n$ （共 k 个 $\log$ ）。

出新的理论模型，证明通过适当增加更新操作的时间复杂度，可以显著减少空间浪费，从而得到更优的时间—空间权衡曲线^[1]。

1.3 研究路线

在阅读上述论文的基础上，用 C++ 做了一个理论落地、简化的原型，把论文里几个关键理论对应到具体模块：整体仍是 Facility → Cubby → Slot；每个 Facility 里有一组 Router；槽里做商压缩；元数据用 MiniArray 存；扩缩容用两张表渐进搬迁。实现时有意牺牲一部分理论上的最优性，换取代码的简洁性、可读性以及易实现性。

本设计关心三个具体的问题：这些模块拼在一起，插入、查询、删除能不能稳定工作；指标（例如踢键次数、路由步数、元数据位数）是否符合预期；和论文里的理想模型相比，简化了哪些地方。本文第 ?? 章回顾论文里描述的算法概念；第三章写本系统的结构与实现；第四章写测试和微基准；最后一章做小结并列出不足、未来展望。

第二章 问题模型与理论基础

2.1 探针复杂度与球槽分配模型

开放寻址哈希表的本质可抽象为经典的球与槽在线分配问题。给定固定数量数组槽位，每个槽仅能容纳一个元素，支持元素动态插入与删除，且元素总数不超过槽位上限。每个键经由哈希函数生成一组固定随机探针序列，元素仅允许放置在自身探针序列的任意候选槽位中。

若元素最终落于探针序列第 i 个位置，则定义其探针复杂度为 $\Theta(1 + \log i)$ ，物理意义为直接定位该元素所需记录位置索引的比特开销，表征哈希表元数据的平均空间代价。在此模型下衍生两个核心优化指标：平均探针复杂度对应哈希表整体额外空间开销，交换成本对应插入删除过程中迁移元素的数量，直接决定操作时间复杂度。哈希表优化的核心目标，即是在探针空间开销与元素迁移时间开销之间实现时空最优折中。

2.2 k-kick 树分层驱逐思想

k-kick 树并非物理意义上的树形存储结构，而是一套面向数组槽位的分层划分与优先级驱逐逻辑框架。其核心思想是将全局槽位划分为 $k + 1$ 个逻辑层级，各层级隔间 Cubby 尺寸随层级加深呈指数级递减，形成由大到小的嵌套分层结构，深层隔间对应元素更偏好的局部存储位置，具备更高存储优先级。

元素在插入前通过随机哈希预先确定所属层级隔间谱系、最大允许存储深度优先级，以及由深层至浅层遍历的专属探针序列。插入过程采用贪心优先级策略，从最深层级向上逐层检索未饱和隔间；若隔间已满但未饱和，则驱逐内部低优先级元素，将当前元素置入空闲槽位，被驱逐元素向上层递归寻找合法位置，形成可控级联驱逐。

由于层级总数被参数 k 限定，单次插入的元素迁移次数被严格约束，以 $O(k)$ 交换成本换取 $O(\log^{(k)} n)$ 的平均探针复杂度，通过有限元素迁移显著压低整体元

数据空间开销。

2.3 紧凑元数据存储核心思想

为在常数时间、近最优空间下管理分层哈希表的路由信息与控制元数据，系统引入两类基础抽象组件：MiniArray 与 Local Query Router。

MiniArray 面向各 Cubby 内部变长元数据条目设计，以内存 B 树结合查表加速思想，实现变长比特条目的紧凑存储与均摊常数时间读写，整体空间开销与所有条目总比特数线性相关，避免固定长度预分配带来的空间冗余。

Local Query Router 面向小规模键集合，基于二进制前缀树压缩编码思想，将键哈希为随机比特串后仅保留区分唯一前缀，整棵前缀树连同槽位元数据被编码为连续比特流。通过批量比特查表跳转，实现键到 Cubby 与内部槽位的常数时间路由映射，且整体编码空间维持对数比特量级。两类组件共同支撑了哈希表在微观元数据存储与宏观键路由上的低空间、常数时间访问能力。

2.4 分层架构整体设计思想

本文所依托的最优哈希表采用 **Facility-Cubby-Slot** 三级分层架构，从宏观组织、局部存储到底层槽位形成完整逻辑体系。

以 Cubby 作为基本管理单元，每个 Cubby 等价于独立的 k-kick 树逻辑实例，内部集成实际数据存储、变长元数据编码以及空闲槽快速检索结构，承载元素放置、驱逐与局部元数据维护能力。上层以 Facility 聚合多个 Cubby，通过顶层路由 MiniArray 与本地路由器完成全局键到设施、再到目标 Cubby 的两级寻址；同时引入多层级 Cubby 与 Tail 尾隔间机制，仅保留尾隔间承载插入删除波动，其余隔间维持满容量不变量，保证系统负载稳定性。

为适配元素规模动态变化，架构引入分布不变量约束各层级 Cubby 的数量配比，通过后台拆分与合并操作平摊重建开销，使扩容、缩容与结构调整均能维持均摊常数时间复杂度。插入、查询、删除等核心操作均遵循统一路由寻址、局部隔间处理、元数据更新与不变量维护的逻辑流程，将底层 k-kick 树规则与上层路由架构有机结合。

2.5 商压缩的信息论最优思想

传统哈希表完整存储原始键的全部比特位，存在固有空间冗余。商压缩的核心思想是利用哈希随机置换的寻址特性，将键的高位比特融合进路由寻址逻辑中隐式存储，无需在数据区重复保存。

通过将键哈希置换后的低位作为设施与路由索引，仅在底层 Cubby 数据区存储键的剩余高位与负载数据；同时在元数据中记录偏好槽位差值与分段中间比特，结合设施编号可逆向完整恢复原始哈希值与真实键。该方式省去路由寻址所依赖的高位比特存储，使单键存储开销逼近信息论下界，叠加元数据与路由的对数级均摊开销，整体哈希表空间复杂度达到理论最优水平。

2.6 算法层次全景总结

整套最优哈希表可归纳为三层逻辑协同架构：

- 微观 Cubby 层：依托 k-kick 树实现元素分层放置与优先级驱逐，配合空闲槽检索与 MiniArray 元数据编码，管控单隔间内的空间开销与元素迁移规则；
- 路由映射层：基于 Local Query Router 与顶层路由数组，实现键到目标隔间与槽位的常数时间寻址，匹配底层探针复杂度的空间约束；
- 宏观 Facility 层：通过多层级 Cubby、Tail 机制与后台重建不变量维护，支撑动态扩容缩容与负载均衡，保证所有操作时间受控于层级参数 k ，整体空间复杂度逼近信息论最优下界。

第三章 系统设计与实现

本章详细介绍系统的落地实现以及与论文理论的差距。

3.1 整体结构

可以把哈希表理解为一个连续的长度为 N 的数组，取 $K = \text{Polylog } N$ ，将这个数组划分为 $\frac{N}{K}$ 个 Facility，每一个 Facility 中包含路由信息和多个 cubby，每个 cubby 有不同数量的 Slot，整个系统可以划分为 Facility $\rightarrow$ Cubby $\rightarrow$ Slot。

Facility 包括：（1）按 tier 分组的多个 Cubby，每个设施中 tier j 的小隔间数量应大致为 $t_j = \frac{(\log^{(j)} n)^2}{(\log^{(j+1)} n^2)}$ ，大小为 $s_j = \frac{K}{(\log^{(j)} n)^2}$ ，tier 的数量偏离目标太远（比如少于 $t_j/2$ 或多于 $3t_j$ ），触发后台重建；（2）tail_tier、tail、tail_owned，约定唯一允许非满的尾 cubby，便于插入策略集中管理；（3）D，长度为 K 的 Router 向量，即前文 D[b]。

Cubby 包括 capacity、size 描述层级与占用；slots 为长度等于容量的 std::optional<uint64_t> 向量，存放商压缩后的 quotient payload ($\pi(k)$ 去掉低 $\log N$ 位的高位)；occupied 记录当前占用槽的下标；free_slots 为空闲槽快速结构；meta 为 MiniArray，与各槽一一对应的变长 MetaEntry 编码（路由与恢复 g_x 所需上下文，见后文）。

Slot 即某个 Cubby 内的整数下标 i ，逻辑上由有序对 (slots[i], meta[i]) 表示：前者为 quotient，后者为可解码元数据。查询校验时从二者恢复 g_x 并与当前 $\pi(k)$ 比较（见后文 slot_pi_matches 路径）。

-
- 1: 给定 facility 编号 r 、其内某 Cubby 指针 C 、槽下标 i
 - 2: Router 层：在 Facility 编号 r 的 D[b] 上 LOCATE(k) $\rightarrow (C^*, i)$
 - 3: Cubby 层：读 C.slots[i] 与 C.meta 中第 i 条元数据
 - 4: Slot 层：若 slots[i] 有值，则结合 meta 调用 RECONSTRUCT_GX_PI 并与 $\pi(k)$ 校验；否则视为空槽
-

3.2 地址拆分与可逆置换

记输入键为 k 。所有操作先计算其置换像 $g_x := \pi(k)$ 。 π 为三轮 Feistel 可逆置换，种子来自 TableParams。记 $g^* = g_x \bmod N$ （实现中 N 为 2 的幂，故等价于取低 $\log N$ 位），再按下式选 facility 与 Router 桶号：

$$g = g_x \wedge (N - 1), \quad r = \lfloor g/K \rfloor, \quad b = g \bmod K.$$

其中 r 为 facility 编号， b 为在该 facility 内选用的 $D[b]$ 。随后在 $D[b]$ 上执行本地路由，得到候选 $(C, slot)$ 。

算法 1 键到 facility 与 Router 桶的拆分（与实现一致）

- 1: $g_x \leftarrow \pi(k)$
 - 2: $g \leftarrow g_x \text{ and } (N - 1)$
 - 3: $r \leftarrow \lfloor g/K \rfloor, \quad b \leftarrow g \bmod K$
 - 4: **return** 在表状态 active 下编号为 r 的 facility 及其 $D[b]$
-

3.3 MiniArray：按槽变长元数据

每个 Cubby 的 MiniArray 保存各槽 MetaEntry 的编码结果。内部维护：各 entry 位长 `bitlens_`、前缀偏移 `offsets_`、未压缩副本 `entries_`、以及打包后的连续位缓冲 `buf_`。单次 update 会改写对应槽的位长与内容，然后调用 `repack()`：先重算所有偏移，再按位把 `entries_` 写回 `buf_`。

算法 2 MiniArray 更新（逻辑层）

- 输入：** 槽号 i ，编码比特序列 $bits$ ，位长 $bitlen$
- 1: $bitlens_[i] \leftarrow bitlen; \quad entries_[i] \leftarrow bits$
 - 2: RECOMPUTE_OFFSETS
 - 3: 按新偏移把各 entry 逐位写入 `buf_`
-

这样实现简单、易与 `bits_total` 指标对接，但单次更新代价与 cubby 槽数同阶，且内存除 `buf_` 外还保留 `entries_` 副本，因此实验里把 `bits_total` 视作**逻辑位数趋势**，不等价于进程真实 RSS。

3.4 Router: 桶内键到槽位映射

各桶内 Router 维护从 key 到 (Cubby*, slot) 的映射。键不直接按原值建树, 而先经 splitmix64 得到 64 位伪随机比特串, 再建 Patricia 式二叉 trie; 内部结点用 split_bit 区分左右子树, 叶结点挂 entry_idx, 哈希冲突时落入 bucket 线性扫描。插入或删除任意键后, 对 trie **整体 rebuild**, 并重新估算编码长度 (内部结点、普通叶、碰撞桶叶的规则不同, 实现里用 BitWriter 走一遍树统计位数)。

算法 3 Router 查询 locate (概要)

```
1:  $bits \leftarrow \text{KEY\_BITS}(k)$ ,  $steps \leftarrow 0$ ,  $u \leftarrow \text{root}$ 
2: while  $u$  为合法结点 do
3:    $steps \leftarrow steps + 1$ 
4:   if 结点为碰撞桶 then
5:     for 桶中每个 entry  $e$  do
6:       if  $e.\text{key} = k$  then return ( $e.\text{value}, steps$ )
7:       end if
8:     end for
9:     return (未命中,  $steps$ )
10:  else
11:    按  $bits$  在 split_bit 处分支进入左或右子
12:  end if
13: end while
```

查询步数上界与 trie 深度相关, 实现中深度不超过约 64; 指标里记为 router_probe_steps 一类字段。

3.5 商压缩、写入与校验

槽内保存 quotient: g_x 去掉低 $\log N$ 位后的高位, 记为 quotient_payload。低位与路由相关信息由 make_meta_entry 根据 N, K 、facility 编号、cubby 容量、槽下标及插入步数等压入 MetaEntry, 再经 encode_meta_entry 写入 MiniArray。写入路径在说明文档中概括为 put_quotient_slot: 同时更新 slots[slot] 与 meta.update。

查询时**不信任**Router 的单一结果: 在候选槽上调用校验例程 (实现为 slot_pi_matches), 从 meta 与 payload 调用 reconstruct_gx_pi 拼回 g_x , 与当前查询的 $\pi(\text{key})$ 比较。恢复完整键时, 再调用 π^{-1} 得到原键, 用于踢出链或迁移扫描。

算法 4 从槽恢复键（与 `recover_key_from_slot` 一致）

```
1: if 槽空 或 meta 解码失败 then return  $\perp$ 
2: end if
3:  $g_x \leftarrow \text{RECONSTRUCT\_GX\_PI}(\text{上下文}, \text{payload}, \text{MetaEntry})$ 
4: if  $g_x$  非法 then return  $\perp$ 
5: end if
6: return  $\pi^{-1}(g_x)$ 
```

3.6 插入主路径与工程版 k-kick

对外 `insert` 在持锁下调用 `insert_no_lock`：若 `active` 与（迁移中的）`old` 上 Router 均已登记该键，则返回幂等“未插入”；否则 `ensure_tail` 保证本 facility 有未满 `tail`，再在 `tail` 上调 `kkick_insert`。类 `KKickTree` 在工程中仅保留 k 的语义，真正行为集中在 `tail` 单 `cubby` 内的有界探针与踢出链：探针下标由 $\pi(\text{cur_key})$ 与步数混合后经 `splitmix64` 对容量取模得到；遇空槽则写入并更新 `MiniArray`、空闲槽树与 Router；遇占用槽则恢复旧键、改写路由，把被踢键作为下一轮 `cur_key`；超过 k 步仍无空位则由 `FreeSlotTree` 任找一空槽兜底。

算法 5 `tail` 内 `kkick_insert`（有界探针 + 踢出链）

输入： 当前 facility、`tail` cubby C 、待插入键 key 、facility 编号 r

```
1:  $cur \leftarrow key$ ,  $moved \leftarrow 0$ 
2: for  $step = 0 \dots k$  do
3:    $pos \leftarrow \text{PROBE\_SLOT}(cur, C.\text{capacity}, step)$ 
4:   if  $C.\text{slots}[pos]$  为空 then
5:     写入 quotient 与 meta；标记占用；Router 登记 ( $cur \rightarrow C, pos$ )
6:     return（成功， $moved \leftarrow step$ ）
7:   else
8:      $kicked \leftarrow \text{RECOVER\_KEY\_FROM\_SLOT}(C, pos, r)$ ；若失败则 return 失败
9:     从 Router 删除  $kicked$ ；用  $cur$  覆盖该槽并更新 meta/Router
10:     $cur \leftarrow kicked$ ,  $moved \leftarrow moved + 1$ 
11:   end if
12: end for
13: 用空闲槽树任取空槽，写入  $cur$  并维护结构； $moved$  递增
```

这与论文中跨多层 cubby 的 k-kick 树不等价，但能支撑功能正确性与 `kick_count`（实现里对应返回的 `moved`）观测。

3.7 查询与删除

查询在 `active` 上按算法 1 定位后 `locate`，再经槽校验；未命中且存在 `old_` 则对旧表重复一遍。删除在定位后清空槽与 `meta`，维护 Router 与元素计数；若删在非 `tail`，则从 `tail` 末尾搬一元素填洞并更新路由，以保持“仅 `tail` 可不满”的习惯。

算法 6 查询命中判定（概要）

- 1: 在 `active` 上算 (f, b) , $(v, steps) \leftarrow D[b].LOCATE(k)$
 - 2: **if** v 为空 **then** 若有 `old` 表则对 `old` 重复；仍空则未命中
 - 3: **end if**
 - 4: **if** `SLOT_PI_MATCHES` $(v, \pi(k))$ **then return** 命中
 - 5: **else return** 未命中
 - 6: **end if**
-

3.8 扩缩容与双表渐进迁移

负载超过 `load_factor`（默认 0.9）时目标容量取 $2N$ ；元素数很少时缩为 $\max\{2, N/2\}$ （不会低于原本的 N ），仍取 2 的幂。触发 `start_migration` 时：把当前 `active` 移入 `old_`，再按新 N, K 建空 `active`；元素计数 $n_$ 不变。之后每次插入/删除末尾调用 `run_migrate_budget`，至多搬迁一个旧 `facility`：不依赖旧 Router，扫描旧 `tail` 与 `tiers` 中仍占用槽，用算法 4 收集键，再按新表规则重新 `kkick_insert`；已在新表出现的键跳过。搬完将该旧 `facility` 清空，推进进度直至 `old_` 释放。

算法 7 单次迁移预算（一个旧 `facility`）

- 1: **if** `old_` 为空 **then return**
 - 2: **end if**
 - 3: 扫描当前进度所指旧 `facility` 内所有占用槽，恢复键集合 K
 - 4: **for** $x \in K$ **do**
 - 5: **if** 新 `active` 上已能路由到 x **then continue**
 - 6: **end if**
 - 7: 按新 N, K 计算 `facility`, `ensure_tail` 后 `KKICK_INSERT`(..., x)
 - 8: **end for**
 - 9: 清空该旧 `facility` 的 `tiers/tail`，推进进度；若全部搬完则释放 `old_`
-

若长时间只有查询、没有更新，迁移预算不会被消耗，旧表会残留——这是实现上的已知限制。

3.9 参数与指标口径

外部参数主要在 TableParams: 规模提示 n (推出初始 $N = \text{next_pow2}(\cdot)$)、 k 、load_factor、Feistel 种子等。内部由 choose_K 等推出 K 、facility 数、cubby 容量上界等, 与《系统实现》中的推导一致, 此处不重复贴常量范围。

运行与压测还会累计 insert_moved_total、router_steps_total、meta_bits_total、resize_start/finish 等。需强调: meta_bits_total 多为“按次操作记录的编码规模累加”, 不等价于真实堆内存; 适合比较**相对趋势**, 不宜直接当作严格空间最优性的数值证据。

第四章 系统验证与实验设计

4.1 功能正确性测试

`otsh_tests` 是系统的自动化回归入口。该程序直接调用 `HashTable`，不经过 HTTP 层，因此更适合检查核心数据结构本身。测试流程包括：

- 初始化哈希表，并检查返回状态与表状态中的 *N*、*Facility* 数量是否有效；
- 使用固定随机种子生成唯一键集合，执行批量插入，要求每个新键返回 `inserted=true`；
- 对全部已插入键执行查询，要求返回 `found=true`；
- 随机删除一半键，要求删除操作成功；
- 再次查询已删除键，要求返回未命中；
- 对剩余键重复插入，要求返回 `inserted=false`，验证幂等语义。

测试程序结束时输出一行 JSON，其中包含 `all_passed`、通过和失败数量、每个 case 的名称与结果，以及 `ops_insert`、`ops_query`、`ops_delete`、`insert_moved_total`、`delete_moved_total` 等指标摘要。若 MySQL 可用，结果追加到 `otsh_test_session_log`；否则写入本地测试日志。这样，功能验证既能由进程退出码判断，也能留下可审计记录。

4.2 性能基准

`otsh_perf` 用于隔离算法热路径。它不链接 MySQL，也不经过 HTTP JSON 编解码，主要测量内存中的插入、顺序命中查询、随机命中查询、删除和混合查询。程序支持命令行参数 `n_insert`、`n_rand_queries`、`table_n_hint`，并通过环境变量 `OTSH_SEED` 控制随机种子。

每个阶段记录操作数、墙钟时间、吞吐、平均延迟与分位数（含 `p50`、`p95`、`p99`）；人类可读结果输出到 `stderr`，单行 JSON 输出到 `stdout`，便于脚本收

集到 perf/results/。表 4-1 给出一组默认参数下的微基准摘要（表中只列平均延迟与 p50），参数为 $n_insert = 5 \times 10^4$ 、 $n_rand_queries = 2 \times 10^5$ 、 $table_n_hint = 2 \times 10^5$ 、 $OTSH_SEED = 1$ ，在本机一次跑出的结果，换机器会有偏差。阅读时主要看相对大小：查询是否明显快于插入、混合查询是否接近纯命中查询等。

表 4-1 otsh_perf 微基准各阶段耗时与延迟摘要

阶段	操作数	墙钟时间/ms	吞吐/s ⁻¹	平均延迟/ns	p50/ns
初始化	—	35.44	—	—	—
插入	50000	1779.45	28099	35455	10700
顺序查询（全命中）	50000	146.23	341916	2802	2700
随机查询（命中）	200000	637.76	313600	2981	2900
删除一半键	25000	162.40	153939	6372	5800
混合查询（命中 + 未命中）	100000	258.93	386199	2242	2600

4.3 指标与数据库观测

操作级指标以 SqlMetricRow 形式产生，核心字段包括 operation_type、probe_count、kick_count、latency_ns、cubby_tier 和 snapshot_id。存储层先把指标放入内存缓冲，再批量写入 otsh_op_metrics，避免高频操作逐条落库造成过大扰动。

这些字段对应第 ?? 章中的理论量。probe_count 可用于观察路由成本，kick_count 可用于观察更新中的交换成本，latency_ns 用于描述工程延迟，cubby_tier 可用于区分不同层级的行为差异。典型查询包括按操作类型统计平均和分位延迟、按 tier 统计平均踢出次数、按快照比较不同参数下的路由步数。

结构快照由 HashTable::visit_structure 驱动。系统把 Facility、Cubby、Slot 和 tier 聚合写入对应表。facility 记录 tail 所在 cubby，cubby 记录 tier、capacity、size 和 is_tail，slot_snapshot 记录槽位是否占用及 $\pi(x)$ 形式的 key hash，tier_stat 记录每个 Facility 在各 tier 上的 cubby 数量。这些表可用于检查 tail 是否唯一、非 tail cubby 是否接近满载、tier 数量是否偏离目标分布。

4.4 可视化与结果呈现

TODO

第五章 总结与展望

5.1 工作总结

本毕业设计参照《On the Optimal Time/Space Tradeoff for Hash Tables》，用 C++ 实现了一个带商压缩、局部路由和双表渐进迁移的开放寻址哈希表原型。对外以 `HashTable` 为入口，内部按 `Facility`、`cubby`、`slot` 组织；用可逆置换做全局打散，用 `Router` 做桶内定位，用 `MiniArray` 存恢复键所需的变长信息。

功能测试覆盖了批量插入、全表查询、随机删一半、删后再查、重复插入等路径；微基准程序则单独量了插入和几类查询的耗时。就手头保存的日志而言，查询延迟在纳秒量级，插入因为要维护元数据和可能触发扩容，会慢不少。这些结果只能说明“这一版实现跑起来大致合理”，不能代替论文里的概率分析。

5.2 不足与局限

与原文相比，差距主要在三点。第一，`k-kick` 并没有做成跨多层 `cubby` 的完整机制，而是缩在 `tail` 里做有界探针和踢出链。第二，`MiniArray` 和 `Router` 为了省事，更新时整段重打包或重建，复杂度和论文里追求的局部更新不同。第三，日志里的 `meta_bits` 等是编码长度的估算，加上 `vector`、指针等常规开销，不能当成严格的空间上界证明。

实验也还偏少：没有系统扫 n 、 k 、负载因子的大网格；没有和 `libc` 哈希表或其他开源实现并排对比。这些留给以后有时间再补。

5.3 可改进方向

若继续在本项目上工作，可以考虑：把 `k-kick` 一侧逐步改成更接近论文描述的分层维护；给 `MiniArray`、`Router` 做增量更新以减轻热点；把性能实验做成可复现的脚本（固定机器说明、多次取中位数）。

参考文献

- [1] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. On the Optimal Time/Space Tradeoff for Hash Tables[EB/OL]. (2021-11-04) [2026-05-02]. <https://arxiv.org/abs/2111.00602>. arXiv: 2111.00602.
- [2] CORMEN T H, LEISERSON C E, RIVEST R L, et al. Introduction to Algorithms [M]. 3rd ed. Cambridge, MA: MIT Press, 2009.
- [3] PAGH R, RODLER F F. Cuckoo Hashing[J/OL]. Journal of Algorithms, 2004, 51(2): 122-144. DOI: 10.1016/j.jalgor.2003.12.002.
- [4] FREDMAN M L, KOMLÓS J, SZEMERÉDI E. Storing a Sparse Table with $O(1)$ Worst Case Access Time[J/OL]. Journal of the ACM, 1984, 31(3): 538-544. DOI: 10.1145/828.1884.

致 谢

行文至此，落笔为终。感谢遇到的导师、感谢身边的每一个同学、感谢遇到的每一个人。凡是过往，皆为序章，很喜欢看到的一句话，人生如棋，落子无悔。愿我们跃入人海，各自灿烂。